digital twin. Once the data has been ingested, we can run analytical queries over it, or use it to retrain a demand prediction ML model for each monthly batch.

Listing 4.4 shows this in action by downloading the June 2025 yellow cab trip data, performing some simple cleansing and loading it to a SQLite database for analysis. Through this analysis we learn that this month nearly four million cab trips were made, with an average distance of 3.6 miles.

Listing 4.4 Batch ingestion of the close to four million yellow cab journeys made in June 2025 for analysis via SQL.

```
import pandas as pd
import sqlite3, requests

db = sqlite3.connect('taxi.db') #1
db.execute('''CREATE TABLE IF NOT EXISTS trips (
    pickup_time TEXT, distance REAL, fare REAL, tip REAL, total REAL)''')

def download(): #2
  url = 'https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_
  ➥2025-06.parquet'
  with open('taxi.parquet', 'wb') as f:
    f.write(requests.get(url).content)

def process(): #3
  df = pd.read_parquet('taxi.parquet')
  clean = df[
    (df['fare_amount'] > 0) & (df['fare_amount'] < 500) &
    (df['trip_distance'] > 0) & (df['trip_distance'] < 100)
  ][['tpep_pickup_datetime', 'trip_distance', 'fare_amount', 'tip_amount',
  ➥ 'total_amount']]

  clean.columns = ['pickup_time', 'distance', 'fare', 'tip', 'total']

  clean.to_sql('trips', db, if_exists='append', index=False) #4
  print(f"Complete: {len(clean):,} trips loaded")

def summarize(): #5
  stats = pd.read_sql_query("""
    SELECT COUNT(*) trips, ROUND(AVG(distance),1) avg_miles,
      ROUND(AVG(fare),2) avg_fare, ROUND(SUM(total)) revenue
    FROM trips""", db)
  print("\n", stats.to_string(index=False))

download()
process()
summarize()
```

#1 Create a SQLite database to store the data.
#2 Download the data for June 20025.
#3 Load the Parquet file to a dataframe and remove records with no distance or fare.
#4 Write the cleansed data to the SQLite database.
#5 Use a SQL query to summarize the data.